翁恺课 · 第 5 主题

W5 循环应用与数组初识 · 练习

对齐翁恺《程序设计入门——C语言》第 5 讲 · 参考答案含「思路 + 常见错法」· 请先独立完成再看答案

填空 W5-1

① 求 n 个数的平均分：累加器 sum 在使用前必须先____（写赋值语句）；否则属于____行为，gcc 在优化开关下会给 ____ 警告（见讲义 5.7 坑①）。

填空 W5-2

② int a[6] 能安全使用的下标范围是 a[____] 到 a[____]；遍历它应写 for (i = 0; i < ____; i++)，写成 i <= 6 会____（越界 / 刚好 / 漏一个）。

填空 W5-3

③ 判断素数用标志变量：先 isprime = ____；循环里一旦 n % i == 0 就 isprime = ____ 并用关键字 ____ 退出循环；循环结束 isprime 仍为 1 则是____。

改错 W5-4

④ 下面程序想算 {1, 2, 3} 的和，但有 1 处逻辑错误导致结果不可靠（可能"碰巧对"）。先自己找，再对照答案（提示：看累加器 sum）。

```c
/* debug_wk5_401.c — 数组求和（找 1 处错误）*/
#include <stdio.h>

int main(void)
{
    int a[3] = {1, 2, 3};
    int sum;            /* ? */
    int i;
    for (i = 0; i < 3; i++) {
        sum += a[i];
    }
    printf("sum = %d\n", sum);
    return 0;
}
```

编程 W5-5

⑤ 读 n 个成绩（n 自定），统计并输出及格（≥60）人数。要求用累加器计数，不许用数组（边读边数）。示例：45、78、59、90、60 → 及格 3 人。

编程 **W5-6**

⑥ 读 n（≤10）个整数存进数组，逆序输出。示例：输入 1 2 3 4 → 输出 4 3 2 1。这题必须用数组（边读边输出做不到逆序）。

专业场景 **W5-7**

⑦ 老师要统计一次测验：读 n 个成绩进数组，输出**最高分、平均分、不及格（<60）人数**。要求同时用上"数组 + 打擂台 + 累加器 + 标志/计数"。示例：78、92、55、61、48 → 最高 92、平均 66.80、不及格 2 人。

概念 **W5-8**

⑧ 结合讲义图 W5-1：`int a[5]` 的内存里到底有几个盒子？写 `a[5] = 100` 会发生什么？为什么 gcc 编译时常常不报错，这叫什么行为？写遍历循环时下标条件要注意什么？

参考答案

① 初始化为 0：`sum = 0;`（或 `int sum = 0;`）。没初始化属于**未定义行为（UB）**，gcc 在优化开关下给 `warning: 'sum' is used uninitialized [-Wuninitialized]`。

思路：累加器必须先"清零"再开始加。常见错法：忘了初始化，结果要么垃圾值，要么碰巧对——但哪怕碰巧对也是 bug，换个环境就炸。

② `a[0]` 到 `a[5]`（共 6 个盒子，下标 0~5）；遍历写 `i < 6`；写成 `i <= 6` 会**越界**（用了不存在的 a[6]）。

思路：下标从 0 开始，所以"小于数组大小"而不是"小于等于"；数组大小是 6，最后一个下标是 5。常见错法：记成从 1 开始，或把 < 写成 <=，都会踩越界。

③ `isprime = 1`；发现整除时 `isprime = 0` 并 `break`；循环结束仍为 1 则是**素数**。

思路：标志变量先"假设成立"，一旦找到反例就翻脸，最后看开关状态。常见错法：break 放错位置、或忘了置 0 只在 printf 判断——标志变量的意义就是"把结论存进变量"。

④ 错误：`sum` 声明后**没初始化**（第 8 行 `int sum;`），直接 `sum += a[i]` 是从垃圾值开始加。修正：改成 `int sum = 0;`。

gcc 诊断（`-Wall -Wextra -O2` 实测原文）：

```
wk5_bug_sum.c:10:13: warning: 'sum' is used uninitialized [-Wuninitialized]
   10 |         sum += a[i];
      |         ~~~~^~~~~~~
```

诊断口径：这是**未初始化**警告（不是语法错误）。**注意**它默认在 -O0 下不报（实测运行甚至"侥幸"输出 sum = 6），所以绝不能靠"运行结果看起来对"来判断代码正确。凡是累加器/计数器，先初始化永远没错。

⑤ 参考代码（wk5_p5_pass.c，gcc 实跑 0 警告）：

```
#include <stdio.h>

int main(void)
{
    int n, i, s, cnt = 0;
    printf("要读几个成绩？");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        printf("第%d个：", i);
        scanf("%d", &s);
        if (s >= 60) cnt++;
    }
    printf("及格人数 = %d\n", cnt);
    return 0;
}
```

输入 45、78、59、90、60，实测输出：

```
要读几个成绩？第1个：第2个：第3个：第4个：第5个：及格人数 = 3
```

思路：这是"累加器"的计数用法——cnt 每遇到一个及格就 +1。常见错法：忘了 cnt = 0 初始化；或把 s >= 60 写成 s > 60（把 60 分的漏掉）。

⑥ 参考代码（wk5_p6_rev.c，gcc 实跑 0 警告）：

```
#include <stdio.h>

int main(void)
{
    int a[10], n, i;
    printf("要读几个数（不超过10）？");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    printf("逆序输出：");
    for (i = n - 1; i >= 0; i--) {
        printf("%d ", a[i]);
    }
    printf("\n");
```

```
    return 0;
}
```

输入 4 个数 1、2、3、4，实测输出：

```
要读几个数（不超过10）？逆序输出：4 3 2 1
```

思路：先全部存进数组（i=0..n-1），再**倒着遍历**（i=n-1..0）。常见错法：倒序循环条件写成 i > 0（漏掉 a[0]）；或数组下标从 1 开始存，倒序又从 1 结束（错位）。

⑦ 参考代码（wk5_p7_grade.c，gcc 实跑 0 警告）：

```
#include <stdio.h>

int main(void)
{
    int a[50], n, i, max, fail = 0, sum = 0;
    printf("学生人数？");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    max = a[0];
    for (i = 0; i < n; i++) {
        sum += a[i];
        if (a[i] > max) max = a[i];
        if (a[i] < 60) fail++;
    }
    printf("最高 = %d，平均 = %.2f，不及格 = %d\n", max, (double)sum / n, fail);
    return 0;
}
```

输入 78、92、55、61、48，实测输出：

```
学生人数？最高 = 92，平均 = 66.80，不及格 = 2
```

思路：一次遍历同时干三件事——累加 sum（求平均）、打擂台 max、计数 fail；求平均要先 (double)sum / n 避免整数除法截断。常见错法：打擂台把 max 初始化为 0（全负分时会错）；或忘了 (double) 导致平均被截成整数。

⑧ int a[5] 内存里有**5 个**盒子，下标 0~4。写 a[5] = 100 是**数组越界**——写到了数组外面那片未知内存。gcc 经常不报，因为这属于**未定义行为（UB）**：编译器不保证拦，也不保证后果——可能写入相邻变量的内存（悄悄改坏别人）、可能读到垃圾值、可能程序崩溃。写遍历循环下标条件必须 i < 数组大小（这里是 i < 5），别用 <=。

> 思路：数组越界是 C 里"编译期最隐蔽、运行期最危险"的坑之一。教训：**编译没警告 ≠ 代码对**；越是下标循环越要数清边界。

本练习册为自写原创变式题（对齐翁恺《程序设计入门——C语言》第 5 讲），仅作课堂伴读；改错题与编程题参考代码均用 gcc -Wall -Wextra 实测（MinGW-W64 gcc 16.1.0），输出为真实运行结果（含 gcc 警告原文）。原视频/教材版权归原作者所有。